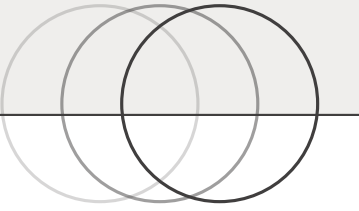


# EXTLINK 網路通訊與 HUFFMAN 壓縮系統

架構設計、通訊協定與核心演算法決策

P1 陳皓祥 / 程式實作：TEAM 1 全體

# 分層設計



傳輸層 (net.c)：管理 TCP Socket 連線與基礎收發

---

封包層 (frame.c)：TL-Frame 封裝，解決 TCP 黏包/半包問題

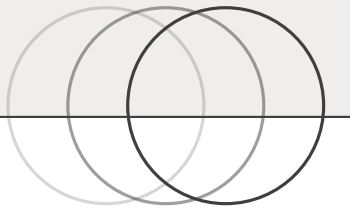
---

驗證與壓縮層 (utf8.c / huffman.c)：執行 RFC 3629 檢查與三模式壓縮

---

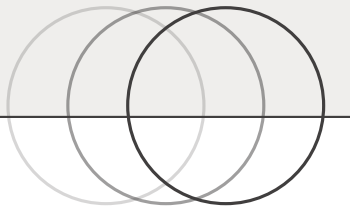
應用層 (chat / transfer)：處理 UI 與檔案系統，無視網路中斷干擾

# TextLink 封包格式協定



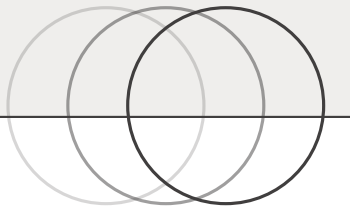
- TL-Frame 結構與跨平台規範
  - Frame 結構：Length (4 bytes) + Type (1 byte) + Payload
  - 端序規範：所有多位元組整數強制轉換為 Big-endian (網路位元組序)
- 檔案傳輸狀態機 (State Machine)
  - FILE\_BEGIN：傳送檔名與檔案總大小 (支援 64MB)
  - FILE\_DATA：將大檔切割，每次最大傳送 64KB payload
  - FILE\_END：傳輸結束驗證與錯誤回報

# Huffman 壓縮區塊設計



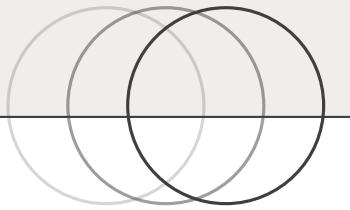
- 「自帶 Codebook」獨立解碼架構
  - 不需要依賴預設字典，接收端可 100% 獨立還原
- 13-Bytes 共通標頭 (Common Header)
  - sym\_type (1B) / orig\_len (4B) / sym\_count (4B) / unique\_syms (4B)
- 三種符號切分模式 (Symbol Types)
  - SYM\_BYTE：通用的單一 byte 切分
  - SYM\_CHAR：精確切割 UTF-8 組合字元 (Code Point)
  - SYM\_S16：解析 WAV Data Chunk，提取 16-bit PCM Sample

# Huffman 決策一：建樹效能與封包最佳化

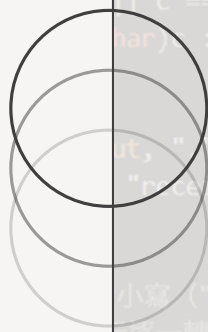


- $O(N \log N)$  Min-Heap 加速與 Tie-breaker
  - 應對 WAV 高達 65,536 種符號，捨棄  $O(N^2)$  迴圈，採用 Min-Heap 建樹
  - 嚴格 Tie-breaker：頻率相同時，以符號 Index 決定優先權，確保跨裝置產生的樹狀結構絕對一致。
- 動態 Codebook 大小與防溢位
  - 若編碼  $\leq 32$  bits，紀錄佔 4 bytes；若  $> 32$  bits，才擴展為 8 bytes。
  - 強制限制 Huffman 樹深  $\leq 64$ ，杜絕 C 語言 uint64\_t 位移溢位崩潰。

# Huffman 決策二：解碼端防護與 WAV 完美邊界



- 動態前綴樹防護 (Prefix Tree Validation)
  - 不依賴頻率，直接讀 Codebook 重建解碼前綴樹
  - 遇到「重複編碼」或「前綴衝突 (Prefix Conflict)」，立刻中止並回傳錯誤
- WAV 奇數長度完美還原
  - 利用  $sz - (sz \% 2)$  將無法成對的 1 byte 移至 Tail 區塊保留
  - 解壓縮後再次呼叫 `parse_wav`，驗證 RIFF 結構是否完美無缺



THANKYOU